Andrey Fritz L. Solijon

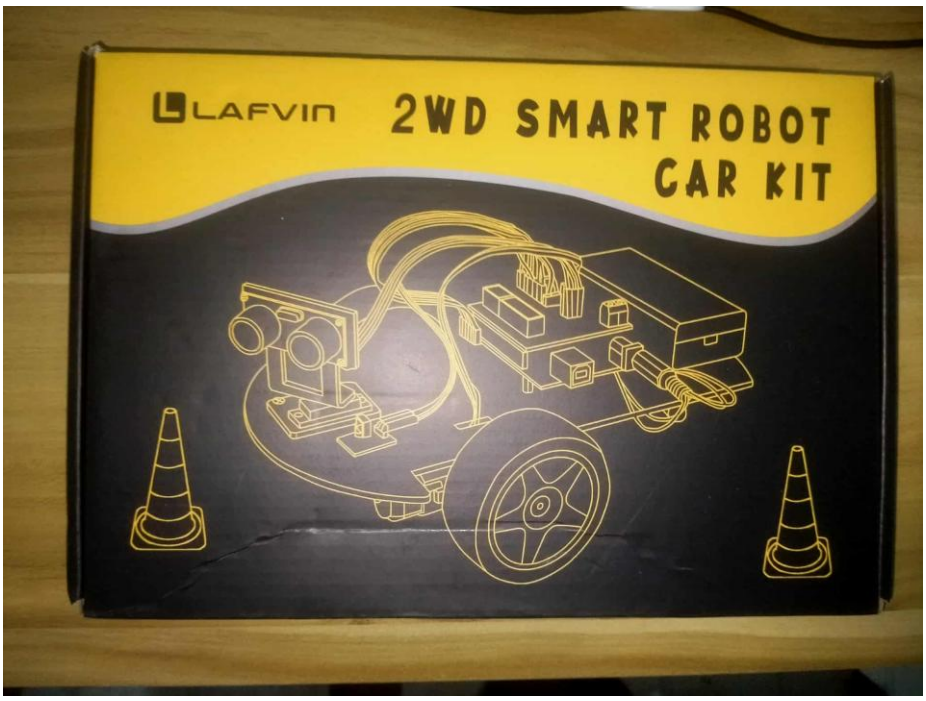
COE185 Capstone Project Milestone 1 Report

Project Title: *Maze Solver and Delivery Robot using Bare-Metal C Programming*

Design Review

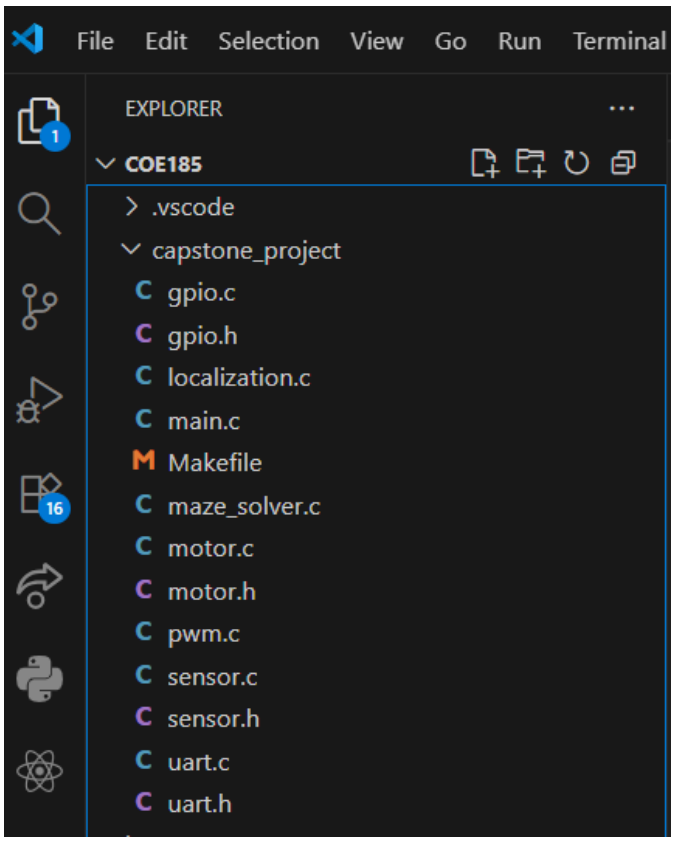
The robot will be capable of navigating through a maze-like environment by detecting obstacles, knowing where to turn, and determining the optimal path to reach a designated destination. At the end of the maze, the robot will deliver the payload and return to the beginning of the maze, simulating delivery tasks in logistics, warehousing, or service robotics. The components that will be used in creating the robot consists of sensors that will detect how far the obstacle is to the robot and some actuators for checking the task status of the robot. This project will also use the bare-metal C programming which will ensure direct control over the microcontroller hardware without relying on external operating systems or abstraction layers. By programming the system at the bare-metal level, the project will highlight resource constrained programming techniques, optimization of tasks, and a deeper appreciation for embedded system design principles.

Materials Update



All of the hardware components like the robot kit, motor driver module and ESP32 have already arrived from online shopping and I will be able to start making the hardware for the project.

Software and GitHub repo (Work in Progress)



```c
#include <avr/io.h>
#include "gpio.h"

// Helper arrays to map enum to actual registers
static volatile uint8_t* ddr_registers[] = {&DDRB, &DDRC, &DDRD};
static volatile uint8_t* port_registers[] = {&PORTB, &PORTC, &PORTD};
static volatile uint8_t* pin_registers[] = {&PINB, &PINC, &PIND};

void gpio_set_direction(GpioPort_t port, uint8_t pin_num, GpioDirection_t direction) {
    if (direction == GPIO_PIN_OUTPUT) {
        *ddr_registers[port] |= (1 << pin_num);
    } else {
        *ddr_registers[port] &= ~(1 << pin_num);
    }
}

void gpio_write(GpioPort_t port, uint8_t pin_num, GpioValue_t value) {
    if (value == GPIO_PIN_HIGH) {
        *port_registers[port] |= (1 << pin_num);
    } else {
        *port_registers[port] &= ~(1 << pin_num);
    }
}
```

```
PS C:\Users\Andrey Fritz\Downloads\COE185>
```

I will utilize the headers and module files from the laboratory activities as it will be a great use implementing these modules for my maze-solving delivery robot and because the wall-following robot as a final output of the activities is closely related to my capstone project in terms of hardware and the modules used.

Improved Software Architecture (FSM)